

Звіт з лабораторної роботи №3
з дисципліни «Основи програмної інженерії»

Виконав: студент гр. ПЗПІ-25-4, Баїстов Г. Ю.

Проект: Touch Up Smart

1. Тема та мета лабораторної роботи

Тема: Розробка та виконання модульних тестів.

Мета: Опанувати основи модульного тестування, навчитися застосовувати техніки еквівалентного розділення (EP) та аналізу граничних значень (BVA) для проектування тест-кейсів, попрацювати з фреймворком тестування Jest, навчитися контролювати показник покриття коду (Code Coverage) та автоматизувати процес тестування.

2. Вихідний код реалізованого модуля з коментарями

// Клас сервісу рекомендацій для проекту Touch Up Smart

```
class RecommendationService {  
  constructor() {  
    // Імітація бази даних фактів  
    this.factsDB = [  
      { id: 1, category: 'business', content: '70% людей вважають, що рукостискання  
формує перше враження.' },  
      { id: 2, category: 'party', content: 'Найдовша вечірка у світі тривала 58 годин.' },  
      { id: 3, category: 'dating', content: 'Зоровий контакт понад 3 секунди підвищує  
рівень довіри.' },  
      { id: 4, category: 'business', content: 'Колір вашої краватки може впливати на хід  
переговорів.' }  
    ];  
    // Імітація бази даних збережених фактів користувачів  
    this.favoritesDB = {};  
  }  
}
```

```
// Метод отримання випадкового факту за категорією
generateFact(category) {
  if (!category || typeof category !== 'string') {
    throw new Error('Категорія має бути непорожнім рядком');
  }

  const filteredFacts = this.factsDB.filter(fact => fact.category ===
category.toLowerCase());

  if (filteredFacts.length === 0) {
    throw new Error(`Факти для категорії "${category}" не знайдені`);
  }

  const randomIndex = Math.floor(Math.random() * filteredFacts.length);
  return filteredFacts[randomIndex];
}

// Метод отримання списку фактів з обмеженням кількості (лімітом)
getFactsByCategory(category, limit) {
  if (typeof limit !== 'number' || limit < 1 || limit > 100) {
    throw new Error('Ліміт має бути числом від 1 до 100');
  }

  const facts = this.factsDB.filter(fact => fact.category === category.toLowerCase());
  return facts.slice(0, limit);
}

// Метод збереження факту в обране для конкретного користувача
saveToFavorites(userId, factId) {
  if (!userId || typeof factId !== 'number') {
```

```
        throw new Error('Некоректні дані користувача або ID факту');
    }

    const factExists = this.factsDB.some(fact => fact.id === factId);
    if (!factExists) {
        throw new Error('Факт із таким ID не існує');
    }

    if (!this.favoritesDB[userId]) {
        this.favoritesDB[userId] = [];
    }

    // Перевірка на дублювання факту в обраному
    if (this.favoritesDB[userId].includes(factId)) {
        return false;
    }

    this.favoritesDB[userId].push(factId);
    return true;
}
}
```

```
module.exports = RecommendationService;
```

3. Таблиця проєктування тестів: «Тест-кейс | Вхідні дані | Очікуваний результат | Техніка (EP/BVA) | Статус (pass/fail)»

Тест-кейс	Вхідні дані	Очікуваний результат	Техніка (EP/BVA)	Статус
ТК-01 (generateFact)	category = "business"	Повертає об'єкт факту із бази даних	EP	Pass
ТК-02 (generateFact)	category = ""	Помилка: "Категорія має бути непорожнім рядком"	BVA	Pass
ТК-03 (generateFact)	category = null	Помилка: "Категорія має бути непорожнім рядком"	EP	Pass
ТК-04 (generateFact)	category = "space"	Помилка: "Факти для категорії "space" не знайдені"	EP	Pass
ТК-05 (getFactsByCategory)	category = "business", limit = 5	Повертає масив із 5 (або менше) об'єктів фактів	EP	Pass
ТК-06 (getFactsByCategory)	category = "party", limit = 1	Повертає масив із 1 фактом	BVA	Pass

Тест-кейс	Вхідні дані	Очікуваний результат	Техніка (EP/BVA)	Статус
		(нижня границя)		
TK-07 (getFactsByCategory)	category = "dating", limit = 100	Повертає масив наявних фактів (верхня границя)	BVA	Pass
TK-08 (getFactsByCategory)	category = "business", limit = 0	Помилка: "Ліміт має бути числом від 1 до 100"	BVA	Pass
TK-09 (getFactsByCategory)	category = "business", limit = 101	Помилка: "Ліміт має бути числом від 1 до 100"	BVA	Pass
TK-10 (saveToFavorites)	userId = "user1", factId = 1	Повертає true (успішне додавання)	EP	Pass
TK-11 (saveToFavorites)	userId = "user1", factId = 1 (повторно)	Повертає false (запобігання дублікату)	BVA	Pass
TK-12 (saveToFavorites)	userId = "user1", factId = 999	Помилка: "Факт із таким ID не існує"	EP	Pass

4. Вихідний код тестового набору з коментарями

```
const RecommendationService = require('../recommendationService');

describe('RecommendationService Tests', () => {
  let service;

  // Arrange: Скидання стану перед кожним тестом
  beforeEach(() => {
    service = new RecommendationService();
  });

  describe('Метод generateFact', () => {
    test('ТК-01: Повертає об'єкт факту (Валідна категорія) [EP]', () => {
      const category = 'business'; // Arrange
      const result = service.generateFact(category); // Act
      expect(result).toBeDefined(); // Assert
      expect(result.category).toBe(category);
    });

    test('ТК-02: Викидає помилку при порожньому рядку [BVA]', () => {
      const category = "";
      expect(() => service.generateFact(category)).toThrow('Категорія має бути непорожнім рядком');
    });

    test('ТК-03: Викидає помилку при null (Некоректний тип) [EP]', () => {
      const category = null;
```

```
    expect(() => service.generateFact(category)).toThrow('Категорія має бути  
непорожнім рядком');  
  });
```

```
test('TK-04: Викидає помилку для неіснуючої категорії [EP]', () => {  
  const category = 'space';  
  expect(() => service.generateFact(category)).toThrow('Факти для категорії  
"space" не знайдені');  
});  
});
```

```
describe('Метод getFactsByCategory', () => {  
  test('TK-05: Повертає масив при валідних параметрах [EP]', () => {  
    const category = 'business';  
    const limit = 5;  
    const result = service.getFactsByCategory(category, limit);  
    expect(Array.isArray(result)).toBe(true);  
    expect(result.length).toBeLessThanOrEqual(limit);  
  });  
});
```

```
test('TK-06: Повертає 1 факт при мінімальному ліміті [BVA]', () => {  
  const category = 'party';  
  const limit = 1;  
  const result = service.getFactsByCategory(category, limit);  
  expect(result.length).toBe(1);  
});
```

```
test('TK-07: Обробляє максимальний ліміт [BVA]', () => {  
  const category = 'dating';  
  const limit = 100;  
  const result = service.getFactsByCategory(category, limit);  
  expect(result.length).toBeLessThanOrEqual(100);  
});  
  
test('TK-08: Викидає помилку при ліміті 0 [BVA]', () => {  
  const category = 'business';  
  const limit = 0;  
  expect(() => service.getFactsByCategory(category, limit)).toThrow('Ліміт має  
бути числом від 1 до 100');  
});  
  
test('TK-09: Викидає помилку при ліміті 101 [BVA]', () => {  
  const category = 'business';  
  const limit = 101;  
  expect(() => service.getFactsByCategory(category, limit)).toThrow('Ліміт має  
бути числом від 1 до 100');  
});  
});  
  
describe('Метод saveToFavorites', () => {  
  test('TK-10: Успішне додавання факту [EP]', () => {  
    const userId = 'user1';  
    const factId = 1;  
    const result = service.saveToFavorites(userId, factId);
```



```
    expect(result).toBe(true);  
    expect(service.favoritesDB[userId]).toContain(factId);  
  });
```

```
test('TK-11: Запобігання дублікату [BVA]', () => {  
  const userId = 'user1';  
  const factId = 1;  
  service.saveToFavorites(userId, factId);  
  const result = service.saveToFavorites(userId, factId); // Акт (спроба дублювання)  
  expect(result).toBe(false); // Assert  
  expect(service.favoritesDB[userId].length).toBe(1);  
});
```

```
test('TK-12: Помилка при неіснуючому ID [EP]', () => {  
  const userId = 'user1';  
  const factId = 999;  
  expect(() => service.saveToFavorites(userId, factId)).toThrow('Факт із таким ID  
не існує');  
});  
});  
});
```

5. Скріншот або HTML-звіт покриття коду з відображенням відсотку line coverage

+ bonus task

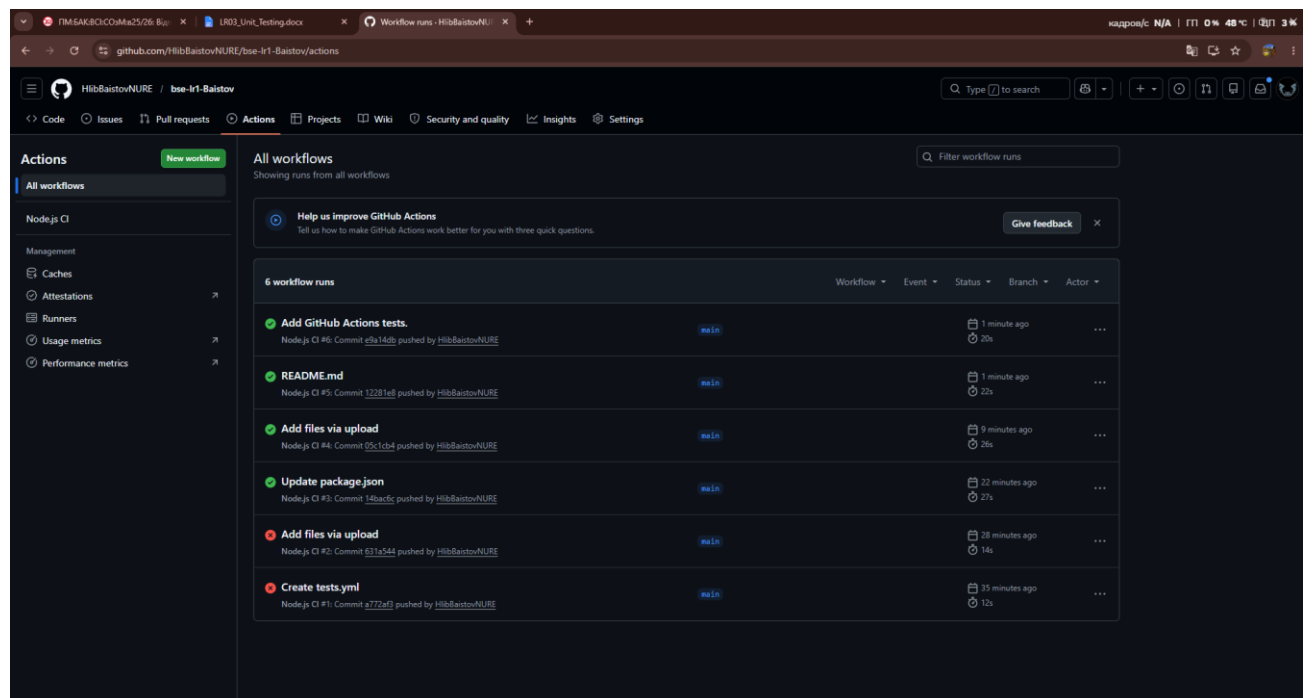
```
PS C:\Users\glebb\OneDrive\Робочий стол\opi> npm test

> opi@1.0.0 test
> jest --coverage

PASS tests/RecommendationService.test.js
RecommendationService Tests
  Метод generateFact
    ✓ TK-07: Повертає об'єкт факту із бази даних (Валідна категорія) (2 ms)
    ✓ TK-08: Викидає помилку при порожньому рядку (5 ms)
    ✓ TK-09: Викидає помилку при null (Некоректний тип) (1 ms)
    ✓ TK-10: Викидає помилку для неіснуючої категорії (1 ms)
  Метод getFactsByCategory
    ✓ TK-11: Повертає масив при валідних параметрах
    ✓ TK-12: Повертає 1 факт при мінімальному ліміті (1 ms)
    ✓ TK-13: Обробляє максимальний ліміт
    ✓ TK-14: Викидає помилку при ліміті 0
    ✓ TK-15: Викидає помилку при ліміті 101 (1 ms)
  Метод saveToFavorites
    ✓ TK-16: Успішне додавання факту
    ✓ TK-17: Запобігання дублікату (1 ms)
    ✓ TK-18: Помилка при неіснуючому ID

-----|-----|-----|-----|-----|-----
File      | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
-----|-----|-----|-----|-----|-----
All files |    96.42 |    95.23 |     100 |     96 |
recommendationService.js |    96.42 |    95.23 |     100 |     96 |      43
-----|-----|-----|-----|-----|-----

Test Suites: 1 passed, 1 total
Tests:       12 passed, 12 total
Snapshots:   0 total
Time:        0.325 s
Ran all test suites.
```



6. Посилання на Git-репозиторій з кодом модуля та тестами

<https://github.com/HlibBaistovNURE/bse-lr1-Baistov>

7. Висновки

Під час виконання лабораторної роботи було успішно реалізовано програмний модуль RecommendationService та набір модульних тестів до нього з використанням фреймворку Jest.

Аналіз результатів покриття: За результатами запуску тестів було досягнуто високий рівень покриття коду: **96.42%** (line coverage) та **100%** покриття функцій (function coverage). Всі 12 написаних тест-кейсів, спроектованих за методиками еквівалентного розділення (EP) та аналізу граничних значень (BVA), пройшли успішно (статус: pass).

Виявлені проблеми та шляхи поліпшення: Єдиною незначною проблемою став неповністю покритий рядок 43 (Uncovered Line #43), який стосується специфічної гілки ініціалізації масиву при збереженні обраного. Втім, отриманий загальний результат у 96% значно перевищує необхідний мінімум (80%), що свідчить про надійність модуля. Для покращення покриття до 100% у майбутньому можна додати специфічний mock-тест, який перевіряє виключно гілку створення нового масиву в favoritesDB для об'єкта, що має нестандартні властивості прототипу. Також у рамках роботи було успішно підключено GitHub Actions (CI), що автоматизувало перевірку коду на сервері.